Reinforcement Learning — Homework 2

NSYSU Drone Control with Reinforcement Learning

Department of Computer Science and Engineering, NSYSU | Reinforcement Learning (Spring 2026)

Course	Reinforcement Learning (Spring 2026)
Assignment	Homework 2 — NSYSU Drone Control
Total Points	100 points
Due Date	2026/05/30 (Sat) 23:59
Submission	Upload zip files (source code, model file, PDF report) to the eLearn system. (Please follow the folder structure)
Others	Individual assignment. Discussion is allowed, but code and report must be your own work.

1. Learning Objectives

In this assignment, you will use the NSYSU Drone simulation environment (ROS 2 + Gazebo Classic) to learn how reinforcement learning (RL) can be applied to a continuous-control problem. Specifically, you are expected to:

- Become familiar with the ROS 2 publisher/subscriber mechanism and the Gazebo physics simulation workflow.
- Understand the trade-offs between PID control and RL-based control in continuous action spaces.
- Formulate a drone control task as a Markov Decision Process (state, action, reward, discount factor).
- Train an RL agent using a modern library and analyze its training curves.
- Communicate your motivation, literature review, and experimental results in a formal report.

2. Environment

This assignment uses the NSYSU Drone simulation environment, a ROS 2 port derived from tum_simulator. It includes a quadrotor model, a PID control plugin (plugin_drone), and the usual sensors (IMU, GPS, sonar, camera).

2.1 Recommended Setup

- Host: Ubuntu 22.04, Docker 24.x
- If you do not have an NVIDIA GPU, you may run a lightweight headless mode (gzserver only) for training; the Gazebo GUI will not be available.
- ROS 2 Iron (default) is recommended. Humble and Rolling are also supported.

2.2 Key ROS 2 Topics

The topics you will most likely use when training an RL agent are listed below:

Topic	Message Type	Purpose
/simple_drone/cmd_vel	geometry_msgs/Twist	Velocity command (action output)
/simple_drone/gt_pose	geometry_msgs/Pose	Ground-truth pose (observation)
/simple_drone/gt_vel	geometry_msgs/Twist	Ground-truth velocity
/simple_drone/takeoff	std_msgs/Empty	Take off (required after each reset)
/simple_drone/reset	std_msgs/Empty	Reset drone pose (end of episode)
/simple_drone/sonar/out	sensor_msgs/Range	Downward range sensor (useful for obstacle avoidance)

3. Part 1: Environment Setup and fly_straight.py (20 points)

The goal of Part 1 is to confirm that you can run the simulator end-to-end and to give you an intuitive feel for the simple position-to-velocity control loop. It is a prerequisite for Part 2.

3.1 Steps

1. Follow the Quick Start section of README.md to build the Docker image and launch the container with run_docker.sh. Native installation is acceptable but the TAs will not debug it for you.
2. Use launch_drone to start Gazebo and RViz, and confirm that the drone model is visible.
3. Open a second terminal and publish a takeoff message so that the drone lifts off.
4. Run fly_straight.py and confirm that the drone flies from the origin to the default target (5.0, 3.0, 2.0).
5. Modify the target point at least three times (for example (-2, 4, 1.5), (0, 0, 3), and (6, -3, 2)) and record the flight behavior for each case.

3.2 What to Submit

- At least one Gazebo screenshot that clearly shows the drone and its target location.
- A terminal log of a successful fly_straight.py run (screenshot or a .txt file).
- A short write-up (about half a page) describing how the behavior changes when you increase or decrease Kp, or the maximum speed. Include this as Appendix A of your Part 2 report.

4. Part 2: Applying RL Algorithms (75 points)

Part 2 is the main component of this assignment. You must define your own RL task in this drone environment, implement and train an agent, evaluate it, and document everything in a report. The provided rl_fly_to_target.py is only a minimal reference; submissions that are a direct copy of this example with no meaningful extension will not receive credit.

4.1 Task Options

You may choose one of the suggested tasks below or propose your own:

- Task A - Precision Hovering and Disturbance Rejection: train the drone to hover at a given (x, y, z) pose and maintain stability when motionDriftNoise is non-zero.
- Task B - Random Target Navigation: generate a different target point in each episode and train a policy that generalizes across targets (an extension of rl_fly_to_target.py).
- Task C - Trajectory Tracking: train the agent to follow a time-varying reference trajectory such as a figure-eight, a circle, or a helix.
- Task D - Autonomous Obstacle Avoidance: add obstacles to the Gazebo world and use sonar or camera inputs so the agent can navigate around them toward a goal.
- Task E - Multi-Waypoint Cruising: given an ordered list of waypoints, train the agent to visit them in sequence while minimizing total time.

4.2 Implementation Requirements

Regardless of which task you choose, your code and model must satisfy the following minimum requirements:

6. Wrap the environment with the Gymnasium interface (inherit gym.Env; implement reset / step, and define observation_space and action_space).
7. Use at least one RL algorithm (PPO, SAC, TD3, or DDPG are recommended; if you use a custom algorithm, explain it in the report).
8. Use a well-justified reward function. Explain the meaning and weight of each term in the report.
9. Episode termination conditions must be explicit (e.g., target reached, timeout, crash, out of bounds).
10. Produce a training curve (reward vs. episode or timestep) and save it as an image included in the report.
11. Provide a test script that, when run by the TAs, loads your trained model and demonstrates the agent's behavior in Gazebo.

4.3 Reflection Report (30 points)

Submit the report as a PDF, 4 to 8 pages (A4, 12 pt font, 1.5 line spacing recommended). LaTeX or Word are both fine. The report must contain the following sections with the point allocation shown below:

Section	Required Content	Points
1. Task Definition and Motivation	Clearly describe the task you want the RL agent to learn in this drone environment (e.g., precision hovering, trajectory tracking, gate traversal, obstacle avoidance, dynamic target tracking), and explain why the task is worth studying.	5
2. Pain Points of Existing Methods	Analyze the limitations of conventional approaches (PID / LQR / MPC / hand-crafted rules) on your chosen task and explain why RL is a plausible alternative.	5
3. Literature Review	At least three papers from the past five years (PPO / SAC / TD3 / DDPG or UAV RL applications are encouraged). Summarize each paper's method, experiments, and how it inspired your design. Use IEEE or APA citation style.	8
4. Proposed Solution	Include: choice of algorithm and rationale, MDP formulation (S, A, R, gamma), network architecture, hyperparameter settings, and the rationale behind your reward design.	7
5. Results and Discussion	Training curves (reward vs. episode), test-time success rate, failure case analysis, and comparison with a baseline such as the P controller in fly_straight.py.	5

Writing Tips

- Do not make vague claims about motivation or pain points. Point out specifically why PID struggles in your chosen task and give an example.
- A literature review is not a summary of abstracts. For each paper, explain what it inspired in your own design.

- Submissions without any failure-case analysis in the discussion will lose points. Honest failure analysis is more valuable than cherry-picked success curves.
- All figures must be produced by you with proper figure numbers and captions. Copying figures from the internet is considered plagiarism.

5. Submission Guidelines

5.1 Required Files

Package everything in a single ZIP archive named HW2_StudentID_Name.zip (for example HW2_M12345678_Wang_Xiaoming.zip). The archive must contain:

12. Your source code (at least the Gym environment, training script, and test script for Part 2).
13. Your trained model file (.zip for SB3 or .pth for PyTorch is recommended). The file name should correspond to your training logs.
14. The reflection report as a PDF, named HW2_StudentID_report.pdf.
15. Part 1 screenshots and logs placed in a subfolder named part1/.
16. A README.md that describes the execution environment, installation commands, how to reproduce your training run, and how to load and test the trained model.

5.2 Suggested Folder Layout

```
HW2_StudentID_Name/
|-- README.md                # How to run
|-- part1/
|   |-- screenshot_01.png
|   |-- screenshot_02.png
|   +-- terminal_log.txt
|-- part2/
|   |-- drone_env.py         # Gym environment
|   |-- train.py             # Training script
|   |-- test.py              # Test script
|   |-- models/
|   |   +-- ppo_drone.zip    # Trained model
|   +-- logs/
|       +-- training_curve.png
+-- HW2_StudentID_report.pdf
```

5.3 Deadline

Submissions are due by **May 30, 2026 (Saturday) 23:59.**

- No delay allowed

5.4 Academic Integrity

- Copying another student's code or report is strictly prohibited. If substantial similarity is detected, both parties receive 0 for this assignment.
- AI tools (ChatGPT, Claude, etc.) may be used as assistants, but you must disclose how they were used in an Acknowledgement section at the end of the report (for example: debugging help, syntax suggestions, brainstorming reward designs). Reports that are entirely AI-generated without personal understanding will be penalized.
- Citations must be accurate. Copying paragraphs from published papers is considered plagiarism.

6. Grading

The assignment is worth 100 points, with up to 5 additional bonus points. The detailed allocation is shown below:

Item	Description	Points	Deliverable
Part 1	Set up the environment and successfully run fly_straight.py. Provide execution evidence (screenshots, terminal logs, Gazebo views).	20	screenshots / log
Part 2 - Code	RL implementation (Gym environment, reward design, training script). Evaluated on quality, readability, and reproducibility of the training pipeline.	30	.py source files
Part 2 - Model	Trained model file (e.g., .zip or .pth) that passes the TA test script and runs in the simulator without version conflicts.	15	.zip / .pth
Part 2 - Report	Reflection report covering task definition, motivation, pain points of existing approaches, literature review, proposed solution, and experimental discussion.	30	.pdf
Bonus	Extra contributions such as multi-target / obstacle avoidance / trajectory tracking / sim-to-real discussion, or quantitative comparison with baselines (PID, open-loop) presented in figures.	5	included in report
Total		**100 (+5)**	

6.1 Part 1 Rubric (20 points)

- Environment is set up successfully and demonstrated: 10 points.
- At least three alternative target points are tested and documented with screenshots: 6 points.
- Written observations on the effect of Kp and max_speed (Appendix A of the report): 4 points.

6.2 Part 2 Code Rubric (30 points)

- Gym environment is correctly structured (reset, step, and space definitions are reasonable): 8 points.
- Reward design is well-reasoned and aligned with the chosen task: 6 points.
- Training script runs out of the box and provides checkpointing and logging: 6 points.
- Test script loads the model and demonstrates agent behavior: 5 points.
- Code readability and quality of comments: 5 points.

6.3 Part 2 Model Rubric (15 points)

- Model file loads cleanly without version conflicts: 5 points.
- Model achieves a reasonable success rate on the fixed test scenario defined for your task: 10 points.

6.4 Part 2 Report Rubric (30 points)

Section-level allocation is given in section 4.3. In addition, overall presentation (layout, clarity of writing, figure quality) may adjust the score by up to plus or minus 2 points.

7. Resources

7.1 Tools and Environment

- NSYSU Drone README and the plugin_drone / pid_controller source files (the basis of this assignment).
- ROS 2 documentation: https://docs.ros.org/en/iron/
- Gazebo Classic tutorials: https://classic.gazebosim.org/tutorials
- Stable-Baselines3 documentation: https://stable-baselines3.readthedocs.io/
- Gymnasium documentation: https://gymnasium.farama.org/

8. Getting Help

If you run into problems, please try to check the Troubleshooting section of README.md.

Good luck, and may your reward curves only go up!